

LinuxISA Superproject

管理方法学与操作手册

面向 Submodule 治理、Strict Gate、Strict Check 与 Test-Driven Bring-up 的
中文手册

适用范围：linux-isa bring-up superproject

版本：2026-03-14

依据文档整合自：superproject methodology、new agent SOP、maintainer repin checklist

目录

1	文档目标	3
2	执行摘要	3
3	Superproject 的职责边界	3
3.1	Superproject 负责什么	3
3.2	Superproject 不负责什么	4
3.3	一个判断准则	4
4	四个控制平面	4
4.1	Topology 平面	4
4.2	Ownership 平面	4
4.3	Validation 平面	5
4.4	Evidence 平面	5
5	Canonical 模块模型	6
6	为什么一定要双 Lane	6
7	Strict Gate 与 Strict Check	6
7.1	Strict Gate 的定义	6
7.2	Strict Check 的两层结构	7
7.2.1	静态严格检查	7
7.2.2	运行时严格检查	7
8	Test-Driven Bring-up 方法	8
9	为什么这种方法能管理复杂系统	8
10	新 Agent 简化 SOP	9
10.1	任务目标	9
10.2	动手前先做什么	9
10.3	先读哪些文件	9
10.4	Scope 规则	9
10.5	真相源优先级	10
10.6	标准 Agent 闭环	10
10.7	最常用命令	10
10.7.1	静态策略检查	10
10.7.2	AVS 合同检查	10
10.7.3	严格跨仓库闭环	10
10.7.4	双 Lane 收敛	11
10.8	按失败面路由到模块	11
10.9	故障分诊顺序	11
10.10	Waiver 与 Stop Condition	11

10.11 Agent Closeout	12
11 Maintainer Repin Checklist	12
11.1 Repin 前置条件	12
11.2 Repin 前要写清楚四件事	13
11.3 工作区准备	13
11.4 静态策略守门	13
11.5 Repin 机械步骤	13
11.6 每个模块必须回答的四个问题	14
11.7 Repin 后的最小 Gate 矩阵	14
11.7.1 默认必跑	14
11.7.2 当严格集成 parity 很重要时	14
11.7.3 当要关闭 release-strict 质量 run 时	14
11.8 按模块给出的最低证明	14
11.9 证据打包检查表	15
11.10 Waiver 纪律	15
11.11 合并判据	15
11.12 推荐 Commit 形态	16
12 日常操作 Runbook	16
12.1 新一轮 Bring-up 周期的最小流程	16
12.2 建议的命令序列	16
13 故障处理顺序	17
14 反模式	17
15 结语	18

1 文档目标

本文将 LinuxISA bring-up superproject 当前的英文方法学文档整理为一份中文手册，目的不是重新发明一套流程，而是把仓库中已经存在的治理结构、验证脚本、gate 纪律和协作约束，压缩成一套适合人和 agent 共同执行的操作方法。

这份手册回答四类问题：

- superproject 到底负责什么，不负责什么；
- 如何管理大量 submodule，避免把集成层变成补丁堆积层；
- 什么叫 strict gate、strict check、test-driven bring-up；
- 新 agent 与 maintainer 在日常工作中各自应该怎么做。

2 执行摘要

LinuxISA superproject 的核心不是“把所有代码放在一个仓库里统一维护”，而是把多个独立模块的 bring-up 结果用统一的 topology、ownership、validation 和 evidence 机制串起来。

整个方法学可以压缩成七条原则：

1. superproject 是协调层，不是实现倾倒区。
2. 叶子模块先修复，superproject 后 repin。
3. 所有可宣称的 bring-up 进展都必须落到 gate、test 或 contract 上。
4. required 且 non-waived 的 gate 必须通过，不能靠口头解释过关。
5. 机器可读证据优先于 markdown 状态页。
6. pin 与 external 双 lane 是严格收敛的基础。
7. 每次改动都必须能回答：谁负责、证明是什么、证据在哪、是否需要 repin。

3 Superproject 的职责边界

3.1 Superproject 负责什么

linux-isa superproject 的职责是跨仓库协调，而不是替代 leaf repo 的实现。它负责：

- 维护 root.gitmodules 中的 submodule 拓扑；
- 维护 docs/ 和 isa/ 中的架构与 bring-up 合同；
- 维护跨模块 gate、strict closure 和 run artifact；
- 维护 gate ownership、waiver ledger 与技能映射；
- 记录被验证过的已知良好 SHA 组合。

3.2 Superproject 不负责什么

它不应该做下面这些事：

- 不把 `compiler/llvm/emulator/qemu/kernel/linux/rtl/LinuxCore/tools/pyCircuit/lib/glibc/lib/musl` 变成“只在 superproject 修补”的影子副本；
- 不用 superproject 层 workaround 掩盖 leaf module 的真实缺陷；
- 不把状态 markdown 当作比 canonical JSON 更高的真相源。

3.3 一个判断准则

如果 bug 明显属于某个 leaf module, 那么修复必须先落在该模块的 owning repo 中; superproject 只负责 repin、闭环验证和证据发布。

4 四个控制平面

LinuxISA 之所以能管理这么多模块, 是因为它把复杂性拆成四个平面, 而不是把所有信息混在一个“进度表”里。

4.1 Topology 平面

Topology 负责回答：代码应该在哪里、哪些路径允许存在、哪些路径禁止复活。
关键文件：

- `AGENTS.md`
- `docs/project/navigation.md`
- `docs/project/repository-flow.md`
- `.gitmodules`
- `tools/ci/check_repo_layout.sh`

核心规则：

- 不新增随机顶层目录；
- 不恢复废弃路径, 例如 `tests/`、`examples/`、`compiler/linux-llvm/emulator/linux-qemu`；
- Linux 生态的 submodule 链接只存在于 `root .gitmodules`。

4.2 Ownership 平面

Ownership 负责回答：谁负责哪个 gate、agent 能改哪些模块、跨模块权限何时允许。
关键文件：

- `docs/bringup/agent_runs/manifest.yaml`
- `docs/bringup/agent_runs/checklists/*.md`

- docs/bringup/agent_runs/waivers.yaml

核心规则：

- 每个 required gate key 都应映射到 checklist owner；
- agent 必须显式声明可操作模块；
- 只有批准过的 cross-module agent 才能跨多个模块工作；
- canonical skill name 是协作合同的一部分。

4.3 Validation 平面

Validation 负责回答：什么算通过、哪些检查是 blocker、strict closure 如何定义。
关键入口：

- tools/regression/strict_cross_repo.sh
- tools/bringup/check_multi_agent_gates.py
- tools/bringup/check_gate_consistency.py
- tools/bringup/check_avs_contract.py
- tools/bringup/check_avs_profile_closure.py
- tools/bringup/run_runtime_convergence.sh

核心规则：

- 结构性静态校验不通过时，运行时结果不可信；
- runtime closure 必须带 lane 和 run-id；
- release-strict closure 需要 pin 和 external 双 lane。

4.4 Evidence 平面

Evidence 负责回答：证据在哪、是否新鲜、是否能重放、是否与状态页一致。
关键工件：

- docs/bringup/gates/latest.json
- docs/bringup/GATE_STATUS.md
- docs/bringup/gates/logs/<run-id>/<lane>/...
- docs/bringup/agent_runs/skills_evolution/latest.json

核心规则：

- 每个 run 都要记录 command、lane、timestamp、result、artifact；
- SHA manifest 必须精确标识测试时的版本；
- 生成的 markdown 视图必须与 canonical JSON 的时间戳一致。

5 Canonical 模块模型

Superproject 的扩展性来自一个简单规则：每个大模块先证明自己局部正确，superproject 再证明这些局部正确的模块在一起仍然正确。

域	Canonical 模块	主要 closure surface
ISA / 文档	isa, docs	contract check、architecture lint、AVS contract
Compiler	compiler/llvm	compile AVS、coverage、tool build closure
Emulator	emulator/qemu	runtime AVS、strict system、opcode sync
Kernel	kernel/linux	initramfs smoke/full boot、vmlinux build closure
Library	lib/glibc, lib/musl	libc stage gates、runtime smoke
RTL	rtl/LinuxCore	lint、cosim、perf floor
Model	tools/pyCircuit	model diff、interface contract
Integration	superproject	strict cross-repo closure、dual-lane parity

这是管理复杂 bring-up 系统的关键：责任边界先明确，再谈集成。

6 为什么一定要双 Lane

LinuxISA 的严格 bring-up 依赖两个 lane：

- **pin**：使用 superproject 当前记录的 submodule SHA；
- **external**：使用当前外部工作树或外部 build。

二者分别回答两个不同问题：

1. 当前 pinned workspace 是否可复现地通过；
2. 下一次 repin 后，系统是否仍然会通过。

只有 external 通过，说明“未来也许能工作”，但 pinned 环境可能还没稳定；只有 pin 通过，说明“当前版本可复现”，但你可能还没有看见下一轮集成回归。strict closure 要求同时回答这两个问题。

7 Strict Gate 与 Strict Check

7.1 Strict Gate 的定义

在 LinuxISA 里，strict 不是口号，而是可执行约束。strict gate 至少意味着：

- required 且 non-waived 的 gate 必须通过；
- waiver 必须显式记录在 waiver ledger；
- lane、run-id、timestamp 是必填上下文；

- 陈旧工件会被拒绝；
- release-strict 模式不允许 blocked-mode shortcut；
- markdown 状态页不能与 canonical JSON 冲突；
- pin 与 external 两个 lane 的 required gate set 必须一致。

7.2 Strict Check 的两层结构

7.2.1 静态严格检查

静态检查先验证结构本身是否可信，例如：

- manifest schema；
- enforcement mode；
- checklist ID 完整性；
- gate owner 覆盖率；
- canonical skill names；
- module scope 合法性；
- waiver 的 phase 绑定和过期时间。

核心命令：

```
python3 tools/bringup/check_multi_agent_gates.py --strict-always
--mode static --manifest docs/bringup/agent_runs/manifest.yaml
--waivers docs/bringup/agent_runs/waivers.yaml --checklists-root
docs/bringup/agent_runs/checklists
```

7.2.2 运行时严格检查

运行时检查验证某个具体 lane/run 是否满足静态策略：

- required runtime row 是否有合法 owner；
- required 且 non-waived row 是否全部为 pass；
- waiver 是否只在 active phase 内有效；
- multi-agent summary JSON 是否存在并与本次 run 对应；
- 证据是否是当前 run 的证据，而不是旧结果。

核心命令形式：

```
python3 tools/bringup/check_multi_agent_gates.py --strict-always
--mode runtime --manifest docs/bringup/agent_runs/manifest.yaml
--waivers docs/bringup/agent_runs/waivers.yaml --checklists-root
docs/bringup/agent_runs/checklists --report
docs/bringup/gates/latest.json --lane pin --run-id <run-id> --out
docs/bringup/gates/logs/<run-id>/pin/multi_agent_summary.json
```


8 Test-Driven Bring-up 方法

LinuxISA 的 bring-up 不以“代码已经写出来”为完成条件，而以“正确的 gate 已经通过并留下了证据”为完成条件。

推荐闭环如下：

1. 从失败的 contract、test、audit 或 gate 开始；
2. 先用最小可复现命令重现问题；
3. 识别 owning domain 和 owning module；
4. 在 leaf module 中修复；
5. 重跑 leaf gate，直到局部证据变绿；
6. 如果 leaf repo 版本变了，在 superproject 中 repin；
7. 再跑 strict cross-repo closure；
8. 发布新的 machine-readable evidence。

这能避免一种常见失控模式：把 superproject 当成“掩盖叶子模块问题的地方”。

9 为什么这种方法能管理复杂系统

当模块很多时，失控往往来自三件事：

- 所有人都能改所有模块，责任边界消失；
- 状态靠文字汇报，而不是靠可执行证据；
- repin 没有 discipline，多个模块被一起“顺手升级”。

LinuxISA 通过下面的设计把系统稳定下来：

1. 用 manifest 和 checklist 把 gate ownership 固化；
2. 用 static/runtime validator 把流程本身也变成 gate；
3. 用 dual-lane 机制把“当前可复现”与“下一次集成风险”分开看；
4. 用 evidence pack 把每次 run 变成可审计对象；
5. 用 repin discipline 保证集成动作总是建立在 leaf proof 之上。

10 新 Agent 简化 SOP

10.1 任务目标

新 agent 的目标不是“把仓库看起来弄绿”，而是：

1. 留在 canonical workspace 和允许的 module scope 内；
2. 找到最小失败点；
3. 优先修复 owning leaf module；
4. 在 leaf proof 变绿后再 repin 和重跑 strict closure；
5. 留下机器可读证据。

10.2 动手前先做什么

从仓库根目录运行：

```
git submodule sync --recursive
git submodule update --init --recursive
bash tools/ci/check_repo_layout.sh
```

如果 layout check 失败，先停下来修 topology，不要继续解释运行时结果。

10.3 先读哪些文件

按下面顺序读取，避免上下文膨胀：

1. AGENTS.md
2. docs/project/navigation.md
3. docs/bringup/agent_runs/manifest.yaml
4. 你负责的 checklist
5. 对应 gate 脚本

如果任务是架构语义相关，再补读 docs/ 或 isa/ 中的合同文档。

10.4 Scope 规则

- 只在声明过的模块内工作；
- 不创建新的顶层目录；
- 不把工作放进禁止路径；
- 不在 superproject 层修一个明显属于 leaf module 的 bug；
- 不把生成的 markdown 当作最高真相。

10.5 真相源优先级

当多个文件相互冲突时，优先级如下：

1. 失败的 test 或 gate command；
2. isa/ 与 docs/architecture/ 中的 canonical contract；
3. 机器可读报告，例如 docs/bringup/gates/latest.json；
4. 生成的 markdown 视图，例如 docs/bringup/GATE_STATUS.md。

10.6 标准 Agent 闭环

1. 用最小 gate 重现问题；
2. 找到 owning domain 和 module；
3. 在 leaf module 内修复；
4. 重跑 leaf gate；
5. 如有必要，在 superproject repin submodule SHA；
6. 运行 strict cross-repo closure；
7. 刷新 machine-readable artifact 与派生视图。

10.7 最常用命令

10.7.1 静态策略检查

```
python3 tools/bringup/check_multi_agent_gates.py --strict-always
--mode static --manifest docs/bringup/agent_runs/manifest.yaml
--waivers docs/bringup/agent_runs/waivers.yaml --checklists-root
docs/bringup/agent_runs/checklists
```

10.7.2 AVS 合同检查

```
python3 tools/bringup/check_avs_contract.py --matrix
avs/linx_avs_v1_test_matrix.yaml
python3 tools/bringup/check_avs_profile_closure.py --matrix
avs/linx_avs_v1_test_matrix.yaml --status
avs/linx_avs_v1_test_matrix_status.json --tier pr
```

10.7.3 严格跨仓库闭环

```
LINUX_GATE_TIER=pr RUN_EXTENDED_CROSS_GATES=1 bash
tools/regression/strict_cross_repo.sh
```

10.7.4 双 Lane 收敛

```
LINUX_GATE_TIER=pr RUN_EXTENDED_CROSS_GATES=1 bash
tools/bringup/run_runtime_convergence.sh --lane pin --run-id <run-id-pin>

LINUX_GATE_TIER=pr RUN_EXTENDED_CROSS_GATES=1 bash
tools/bringup/run_runtime_convergence.sh --lane external --run-id
<run-id-external>
```

10.8 按失败面路由到模块

失败面	首先进入的模块
compile AVS、coverage、LLVM binutils	compiler/llvm
runtime AVS、strict system、opcode sync	emulator/qemu
Linux smoke/full boot、vmlinux build	kernel/linux
musl/glibc runtime 或 build closure	lib/musl, lib/glibc
cosim、stage/connectivity、perf floor	rtl/LinxCore
model diff、interface contract	tools/pyCircuit
contract docs、architecture lint、AVS contract	isa, docs
lane parity、gate consistency、evidence packaging	superproject

10.9 故障分诊顺序

始终按这个顺序分诊：

1. topology failure；
2. static policy failure；
3. leaf-module gate failure；
4. integration closure failure；
5. evidence freshness 或 report mismatch。

不要在 leaf gate 还是红的时候先调 dual-lane parity。

10.10 Waiver 与 Stop Condition

waiver 只有满足下面条件时才合法：

- 显式记录；
- phase-bound；
- time-limited；

- 出现在 docs/bringup/agent_runs/waivers.yaml。

当出现以下情况时，agent 应停止并升级：

- 需要越过已声明的 module scope；
- 唯一 workaround 会违反架构合同；
- 两个 canonical source 明显冲突，且无法判断谁是 stale；
- 用户已有修改与当前修复直接冲突。

10.11 Agent Closeout

宣布完成前至少确认：

1. 最小相关 leaf gate 已通过；
2. 如果涉及集成，strict cross-repo closure 已通过；
3. artifact path 存在；
4. 已明确给出 repin / no-repin / blocked-with-waiver 决策；
5. markdown 视图是从 canonical report 生成，而不是手改。

11 Maintainer Repin Checklist

11.1 Repin 前置条件

在开始更新 submodule SHA 前，应确认：

- leaf-module 改动已经存在于 owning repository；
- leaf owner 已跑过该模块的核心 gate；
- repin 原因明确；
- 预期会影响哪些 gate 是已知的。

错误的 repin 方式：

- “上游都变了，所以一起 bump 一下。”

正确的 repin 方式：

- “QEMU strict-system timer 修复已落地，因此 repin emulator/qemu，然后重跑 QEMU 和 integration closure。”

11.2 Repin 前要写清楚四件事

在编辑 SHA 前, maintainer 应能写下:

1. 哪些模块将被 repin;
2. 每个模块为什么变化;
3. 哪些 gate 预计会受影响;
4. 这是 leaf-only 还是 cross-module 变更。

如果这四个问题答不清, 说明 repin batch 过于模糊。

11.3 工作区准备

```
git submodule sync --recursive
git submodule update --init --recursive
bash tools/ci/check_repo_layout.sh
```

新的 bring-up 周期建议先同步 canonical skills:

```
bash tools/bringup/sync_canonical_skills.sh --pull-latest
```

11.4 静态策略守门

```
python3 tools/bringup/check_multi_agent_gates.py --strict-always
--mode static --manifest docs/bringup/agent_runs/manifest.yaml
--waivers docs/bringup/agent_runs/waivers.yaml --checklists-root
docs/bringup/agent_runs/checklists
```

如果这一步失败, 不要继续信任后面的 runtime 结果。

11.5 Repin 机械步骤

原则上只更新 intended modules。通用形式如下:

```
git submodule update --remote compiler/llvm emulator/qemu
kernel/linux rtl/LinuxCore tools/pyCircuit lib/glibc lib/musl
workloads/pto_kernels
```

实际执行时, 应尽量收窄到这次确实需要 repin 的模块。
更新后立刻检查:

```
git status
git diff --submodule
```

确认:

- 只有 intended submodule 发生 SHA 移动;
- .gitmodules 只在 URL 策略确有变化时才改;
- 没有混入无关工作区噪音。

11.6 每个模块必须回答的四个问题

对每个 repinned module, maintainer 至少要回答：

1. 哪个 leaf gate 证明了新 SHA 是健康的；
2. 哪个 cross-repo gate 可能因为这次 SHA 移动而回归；
3. 这次 repin 只需验证 pin, 还是必须验证 pin 与 external；
4. 是否有 waiver 需要新增、移除或自然过期。

11.7 Repin 后的最小 Gate 矩阵

11.7.1 默认必跑

```
python3 tools/bringup/check_avs_contract.py --matrix
avs/linux_avs_v1_test_matrix.yaml
python3 tools/bringup/check_avs_profile_closure.py --matrix
avs/linux_avs_v1_test_matrix.yaml --status
avs/linux_avs_v1_test_matrix_status.json --tier pr
LINUX_GATE_TIER=pr RUN_EXTENDED_CROSS_GATES=1 bash
tools/regression/strict_cross_repo.sh
```

11.7.2 当严格集成 parity 很重要时

```
LINUX_GATE_TIER=pr RUN_EXTENDED_CROSS_GATES=1 bash
tools/bringup/run_runtime_convergence.sh --lane pin --run-id <run-id-pin>
```

```
LINUX_GATE_TIER=pr RUN_EXTENDED_CROSS_GATES=1 bash
tools/bringup/run_runtime_convergence.sh --lane external --run-id
<run-id-external>
```

11.7.3 当要关闭 release-strict 质量 run 时

```
python3 tools/bringup/check_gate_consistency.py --report
docs/bringup/gates/latest.json --progress docs/bringup/PROGRESS.md
--gate-status docs/bringup/GATE_STATUS.md --libc-status
docs/bringup/libc_status.md --profile release-strict --lane-policy
external+pin-required --trace-schema-version 1.0 --max-age-hours 24
```

11.8 按模块给出的最低证明

模块	期望的最低证明
compiler/llvm	compile AVS、coverage；若关系到 Linux/libc，还要检查 辅助 LLVM tool closure
emulator/qemu	strict system、runtime AVS、opcode sync、pinned binary build
kernel/linux	vmlinux build、initramfs smoke/full boot

模块	期望的最低证明
lib/glibc	当前 profile 下适用的 G1a/G1b stage gate
lib/musl	build closure 与 runtime smoke
rtl/LinuxCore	当前 phase 下对应的 lint/cosim/perf gate
tools/pyCircuit	interface contract 与 model diff
isa, docs	contract check 和相关生成工件刷新

11.9 证据打包检查表

在合并 repin 前，maintainer 应能指向：

- run ID；
- lane；
- 精确命令；
- 日志路径；
- docs/bringup/gates/latest.json 中的 SHA manifest entry；
- 必要时刷新过的状态页；
- strict runtime closure 对应的 multi-agent summary JSON。

如果这些项目缺失，repin 就不应合并。

11.10 Waiver 纪律

不要通过修改 prose 或依赖隐性共识，把一个红 gate 偷偷带过 repin。
若确实需要 waiver，必须：

- 记录到 docs/bringup/agent_runs/waivers.yaml；
- 绑定 active phase；
- 指定 owner 和 expires_utc；
- 确保 runtime validator 能看见它。

如果 waiver 已经不需要了，应在同一维护窗口中移除。

11.11 合并判据

repin 只有在下面条件满足时才算 ready：

- 本批次只包含 intended modules 的 SHA 变化；
- required 且 non-waived 的 gates 全绿；
- 证据新鲜且机器可读；

- 目标 closure level 所要求的 lane policy 已满足；
- 生成的 markdown 与 canonical JSON 一致；
- commit message 清楚描述了模块范围与 repin 原因。

11.12 推荐 Commit 形态

推荐：

- chore(submodule): repin emulator/qemu for strict-system timer fix
- chore(submodules): repin compiler/llvm and kernel/linux for pinned build closure

避免：

- update deps
- sync repos
- latest upstream

12 日常操作 Runbook

12.1 新一轮 Bring-up 周期的最小流程

1. 同步 submodule 并验证 layout；
2. 同步 canonical skills；
3. 运行 static strict checks；
4. 跑最小失败 gate；
5. 回到 owning leaf module 修复；
6. 如有必要 repin 子模块 SHA；
7. 跑 PR-tier strict closure；
8. 如有必要跑 dual-lane convergence；
9. 最后跑 consistency check 并刷新状态页。

12.2 建议的命令序列

```
git submodule sync --recursive
git submodule update --init --recursive
bash tools/ci/check_repo_layout.sh
```

```
bash tools/bringup/sync_canonical_skills.sh --pull-latest
```

```
python3 tools/bringup/check_multi_agent_gates.py --strict-always
--mode static --manifest docs/bringup/agent_runs/manifest.yaml
--waivers docs/bringup/agent_runs/waivers.yaml --checklists-root
docs/bringup/agent_runs/checklists
```

```
python3 tools/bringup/check_avs_contract.py --matrix
avs/linx_avs_v1_test_matrix.yaml
python3 tools/bringup/check_avs_profile_closure.py --matrix
avs/linx_avs_v1_test_matrix.yaml --status
avs/linx_avs_v1_test_matrix_status.json --tier pr
```

```
LINUX_GATE_TIER=pr RUN_EXTENDED_CROSS_GATES=1 bash
tools/regression/strict_cross_repo.sh
```

13 故障处理顺序

当整个系统出错时，推荐修复顺序如下：

1. topology failure;
2. static policy failure;
3. leaf gate failure;
4. integration closure failure;
5. report freshness / evidence mismatch。

这条顺序非常重要。很多集成层错误只是 leaf regression 的二次症状，如果顺序反了，会浪费大量排障时间。

14 反模式

以下做法会迅速破坏大型 bring-up 项目的可维护性，应避免：

- 手改 GATE_STATUS.md 而不更新 canonical JSON；
- 用 superproject workaround 遮盖 leaf module bug；
- 引入平行但“看起来差不多”的非 canonical 路径；
- 把多个无关 repin 混在一个 speculative batch 里；
- 把 waiver 当成永久政策；
- 在 strict closure 需要双 lane 时只跑一个 lane；
- 让没有明确 module scope 和 checklist ownership 的 agent 参与关键闭环。

15 结语

LinuxISA 能够把编译器、模拟器、Linux、RTL、模型、libc 等多个模块组织成一个可 bring-up、可收敛、可维护的复杂系统，依赖的不是“更多会议”和“更多状态汇报”，而是下面这套可执行结构：

- contract 放在 isa/ 和 docs/；
- topology 放在 root policy 与 .gitmodules；
- ownership 放在 manifest.yaml 和 checklist；
- integration truth 放在 strict gate；
- release truth 放在 canonical JSON evidence。

这也是它对 agent 友好的地方：agent 不需要依赖大量口口相传的 tribal knowledge，只要仓库已经清楚表达：

- 你能改哪里；
- 你要证明什么；
- 失败归谁负责；
- 哪个 artifact 才是权威证据。